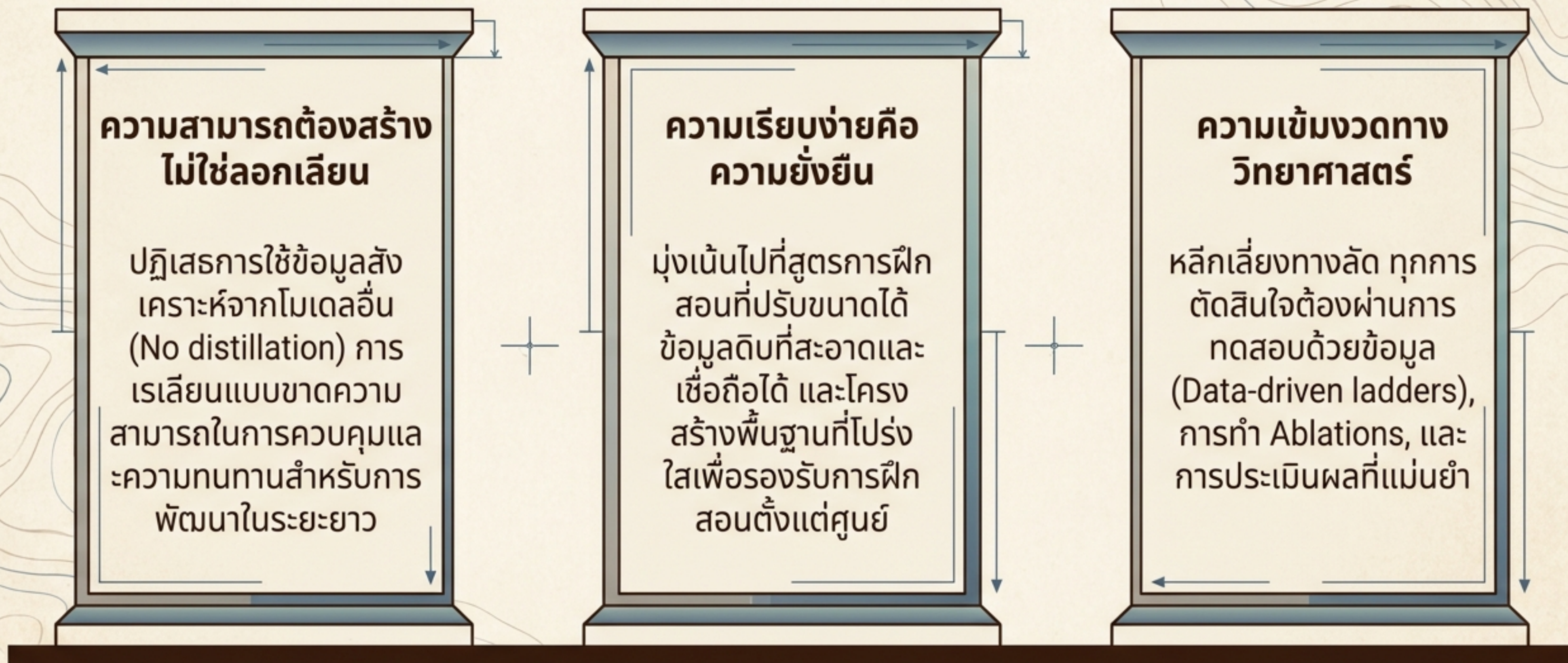


# MAI-Thinking-1: การสร้างกลไกโต้ตอบ

สถาปัตยกรรมเชิงประจักษ์เบื้องหลังโมเดลการให้เหตุผล  
35B พารามิเตอร์จาก Microsoft AI



# ปรัชญาหลักในการออกแบบ

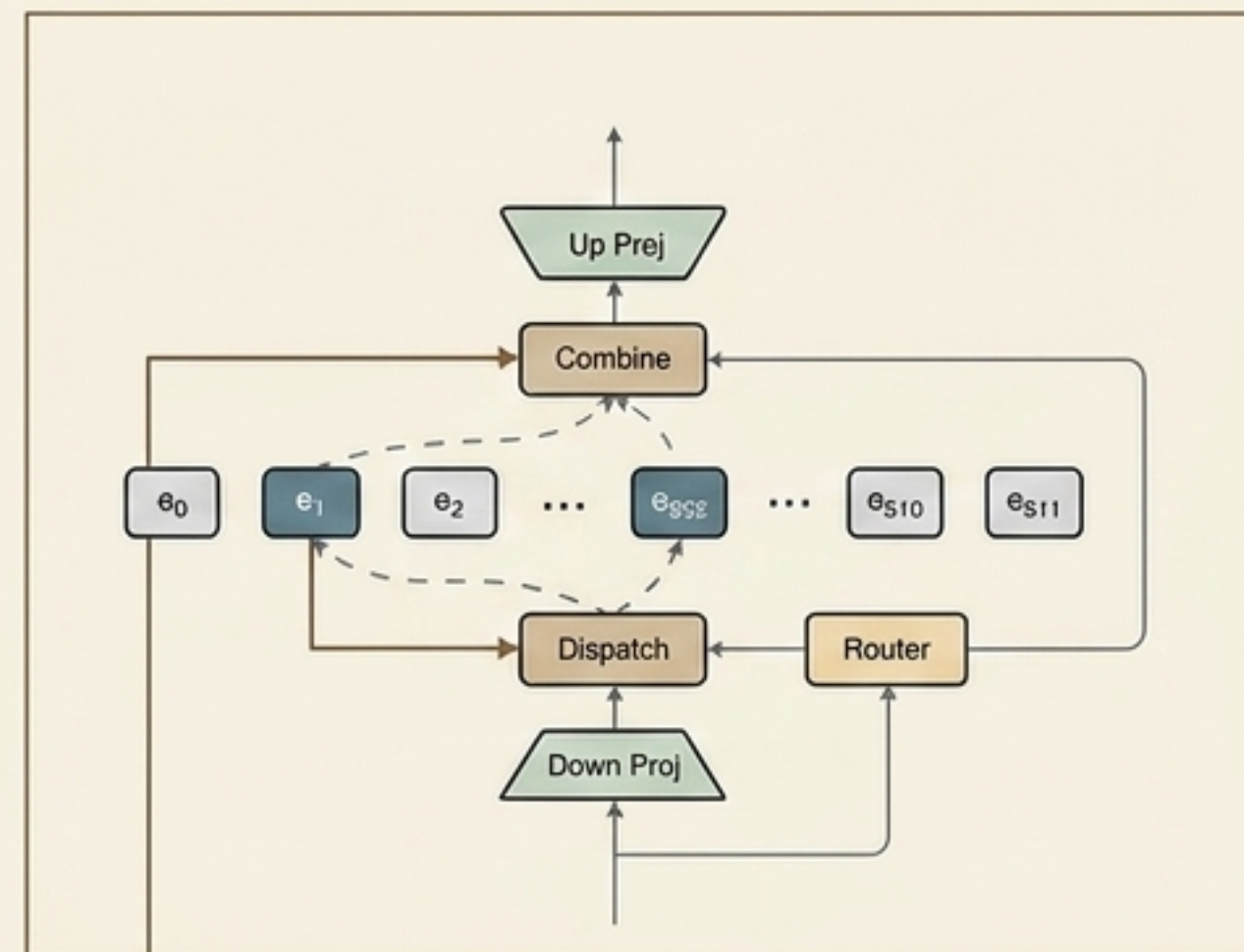
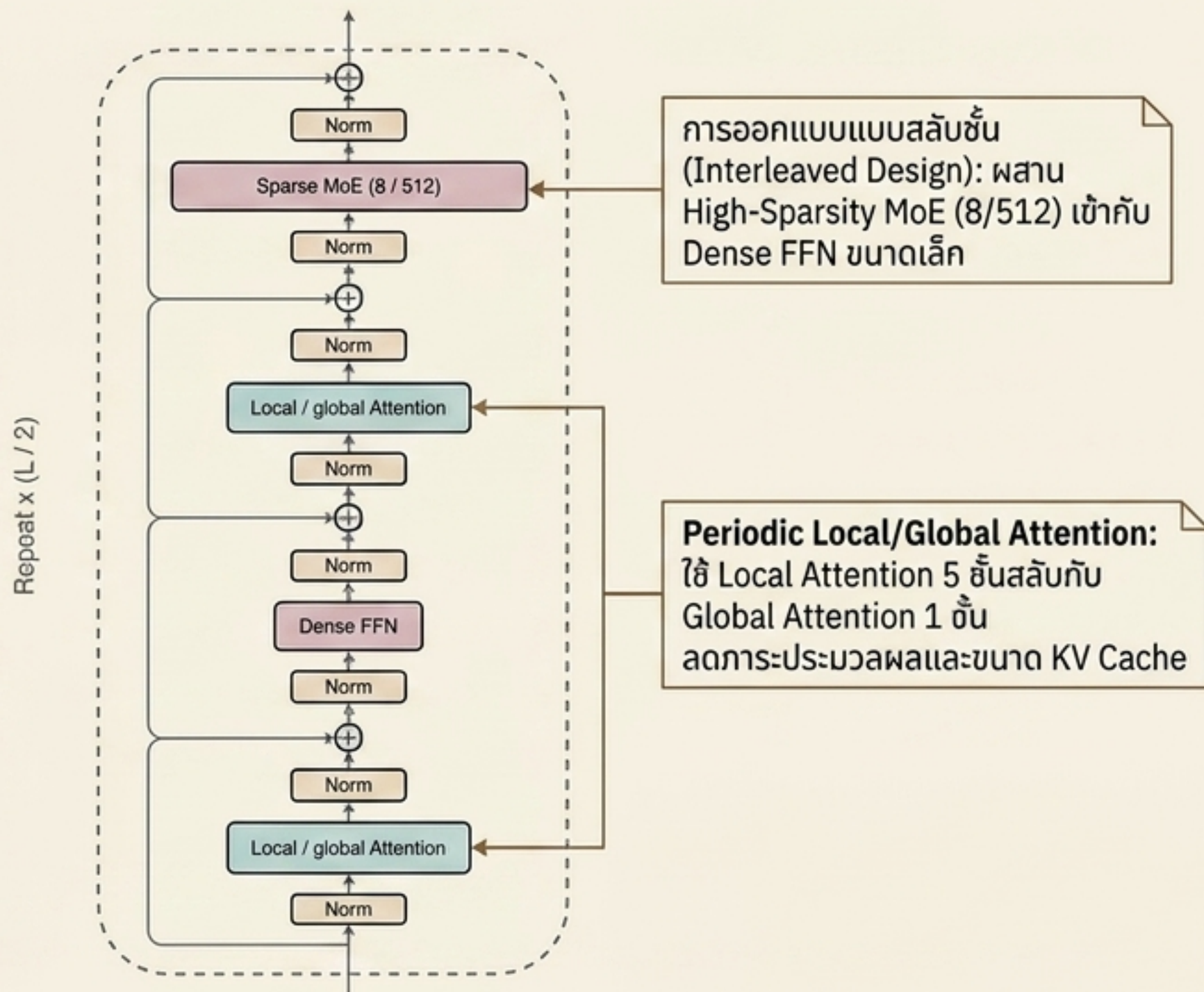




# สถาปัตยกรรม MAI-Base-1: ออกแบบเพื่อการขยายสเกล

Specs

พารามิเตอร์ทำงาน (Active) 35B / รวม (Total) 1T  
| 78 Layers | 256K Context





# เครื่องยนต์ประเมินผล: เหตุใดจึงต้องใช้ NLL?

## การประเมินแบบดั้งเดิม - Traditional Q&A



- ปัญหา: ต้นทุนสูง, ไวต่อรูปแบบ (Formatting quirks) เช่น `\n` vs `\r\n`, ต้องใช้โมเดลผู้ตัดสิน (Judge model)

## การประเมินด้วย Negative Log-Likelihood - NLL

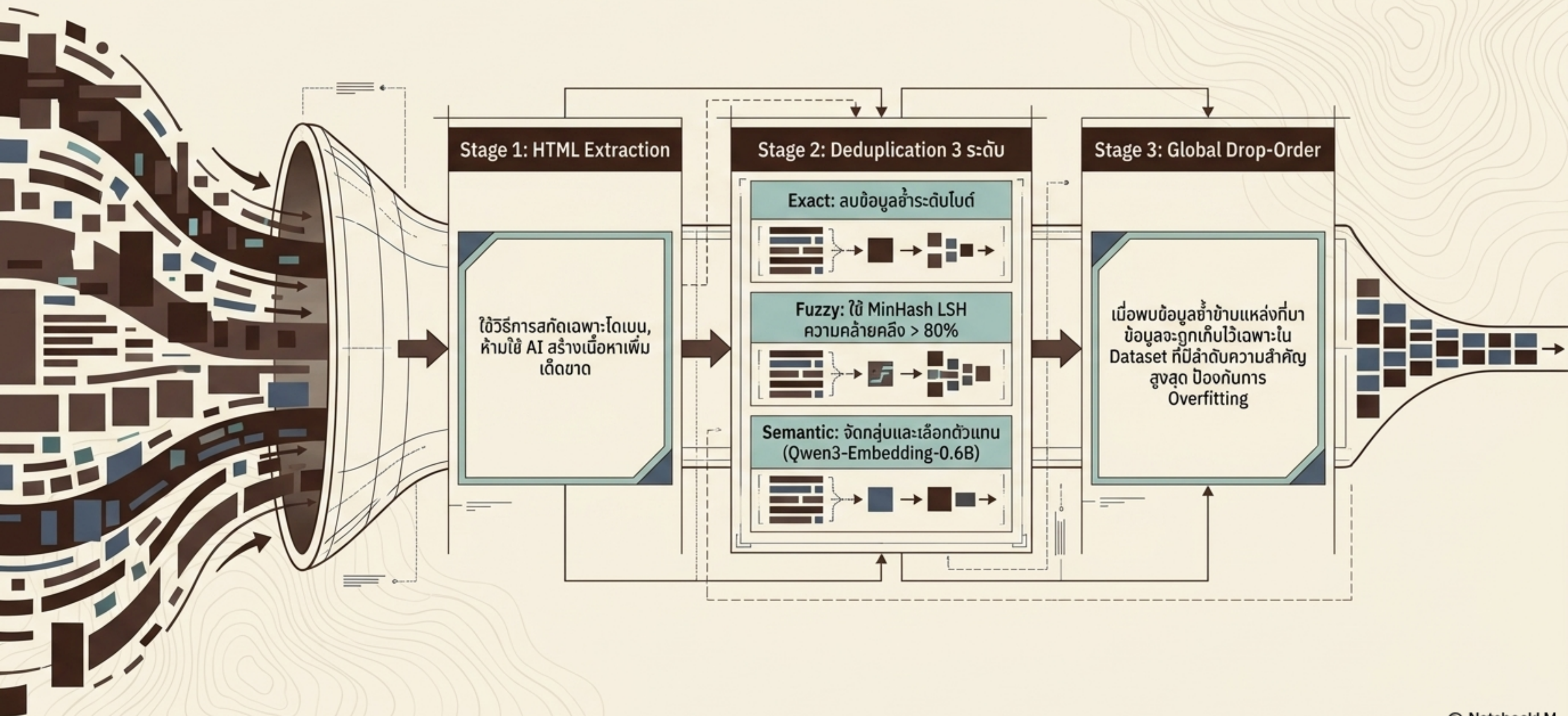


- ข้อได้เปรียบ: ต้นทุนต่ำและรวดเร็ว, ไม่โดนรบกวนจากรูปแบบ, ให้สัญญาณที่เสถียรและแม่นยำสำหรับการปรับสมดุลโมเดล

ตัวชี้วัด **Efficiency Gain (EG)**: มาตรวัด EG ช่วยคำนวณว่า Baseline model ต้องใช้ต้นทุนเพิ่มขึ้นเท่าใดจึงจะเทียบเท่าโมเดลใหม่ (เช่น  $EG = 1.3$  หมายถึงต้องใช้ FLOPs เพิ่ม 30%)



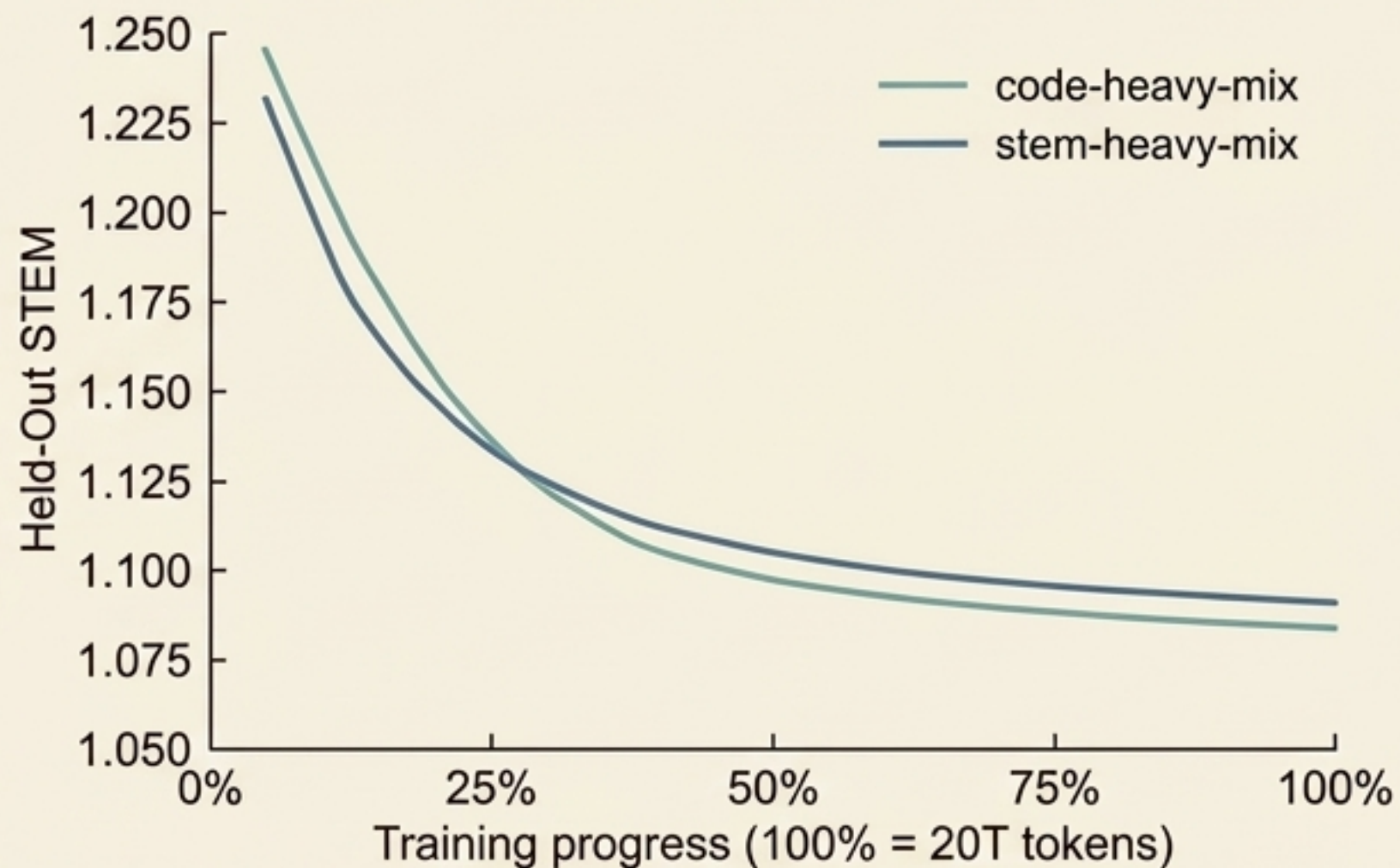
# การสกัดเชื้อเพลิง: ท่อส่งข้อมูล Pre-training





# ศิลปะแห่งการผสมข้อมูล และปรากฏการณ์ Rank Non-Invariance

ปรากฏการณ์ข้ามสเกล: Rank Non-Invariance



**ปรากฏการณ์ข้ามสเกล:** อัตราส่วนข้อมูลที่ชนะในโมเดลขนาดเล็ก (เช่น STEM-heavy) อาจแพ้เมื่อเพิ่มสเกลให้ใหญ่ขึ้น (Code-heavy ชนะที่ 23B)

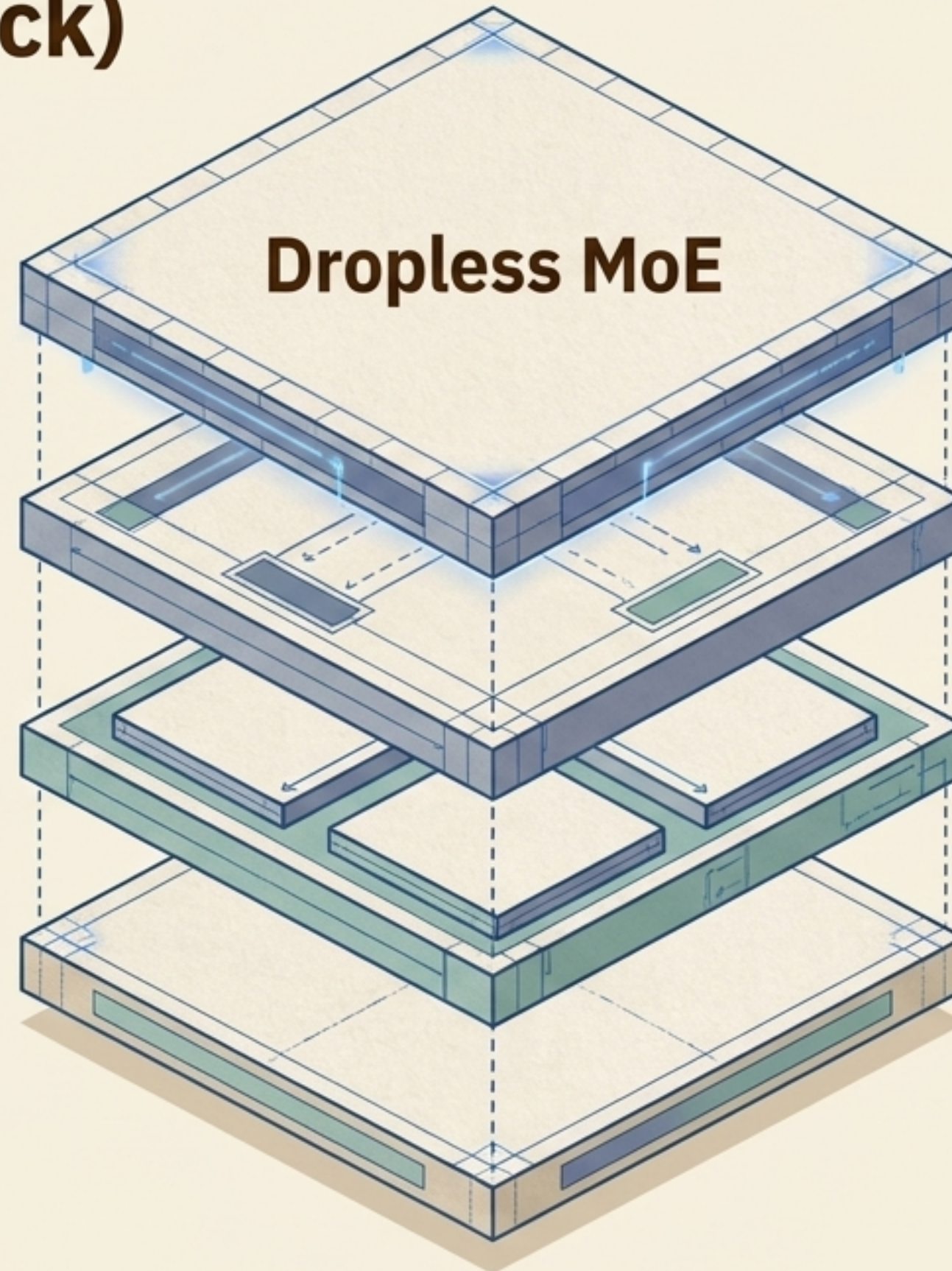
## สัดส่วนข้อมูล 30T Tokens - องค์ประกอบหลัก

|          |       |                                                              |
|----------|-------|--------------------------------------------------------------|
| Code     | 54.6% | เฉลี่ย 2.22 Epochs                                           |
| STEM     | 15.8% | เฉลี่ย 2.17 Epochs                                           |
| Web Text | 14.9% | เฉลี่ย 0.55 Epochs                                           |
| Math     | 5.4%  | เฉลี่ย 5.28 Epochs -<br>หมุนเวียนสูงสุดเพื่อเน้นการให้เหตุผล |
| PDFs     | 4.7%  | เฉลี่ย 0.53 Epochs                                           |



# YOLO: โครงสร้างพื้นฐานแบบปรับแต่งพิเศษ (Custom Infra Stack)

Optimization Topography



**Asynchronous Distributed  
Checkpointing**

**ZeRO-2 / ZeRO-3 &  
Context Parallelism**

**Custom FP8 Kernels**

ขจัดปัญหาการทิ้ง Token  
ด้วยการรองรับ Variable-size  
All-to-all communication

ใช้พื้นที่เก็บข้อมูลส่วนกลางแบบ  
Key-value store เพื่อลดเวลา  
Downtime

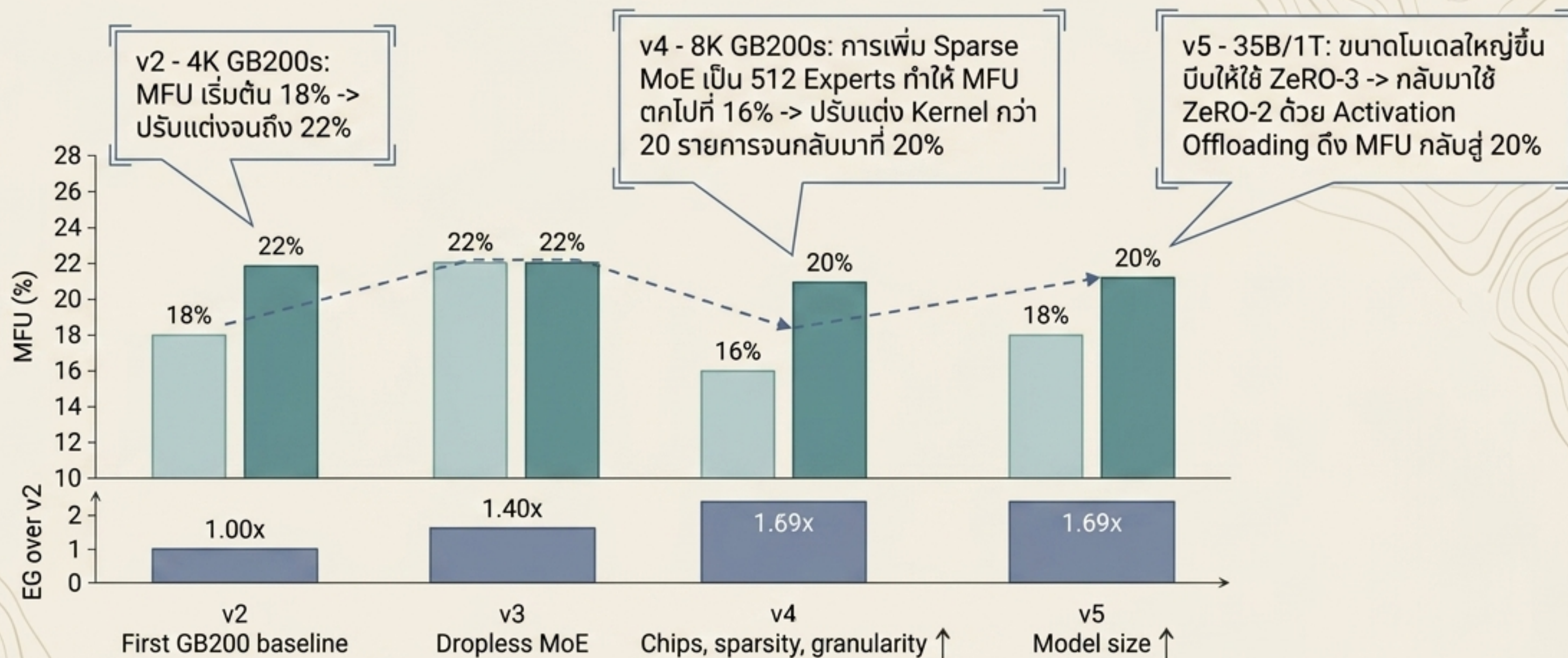
แยกส่วนพารามิเตอร์และ Activation  
Offloading ไปยัง Host Memory

การดำเนินการ Delayed Scaling  
และ Bitwise Determinism แบบสม  
บูรณ์แบบเพื่อการทำซ้ำได้ระดับบิต



# การเพิ่มประสิทธิภาพระหว่าง MFU และ EG

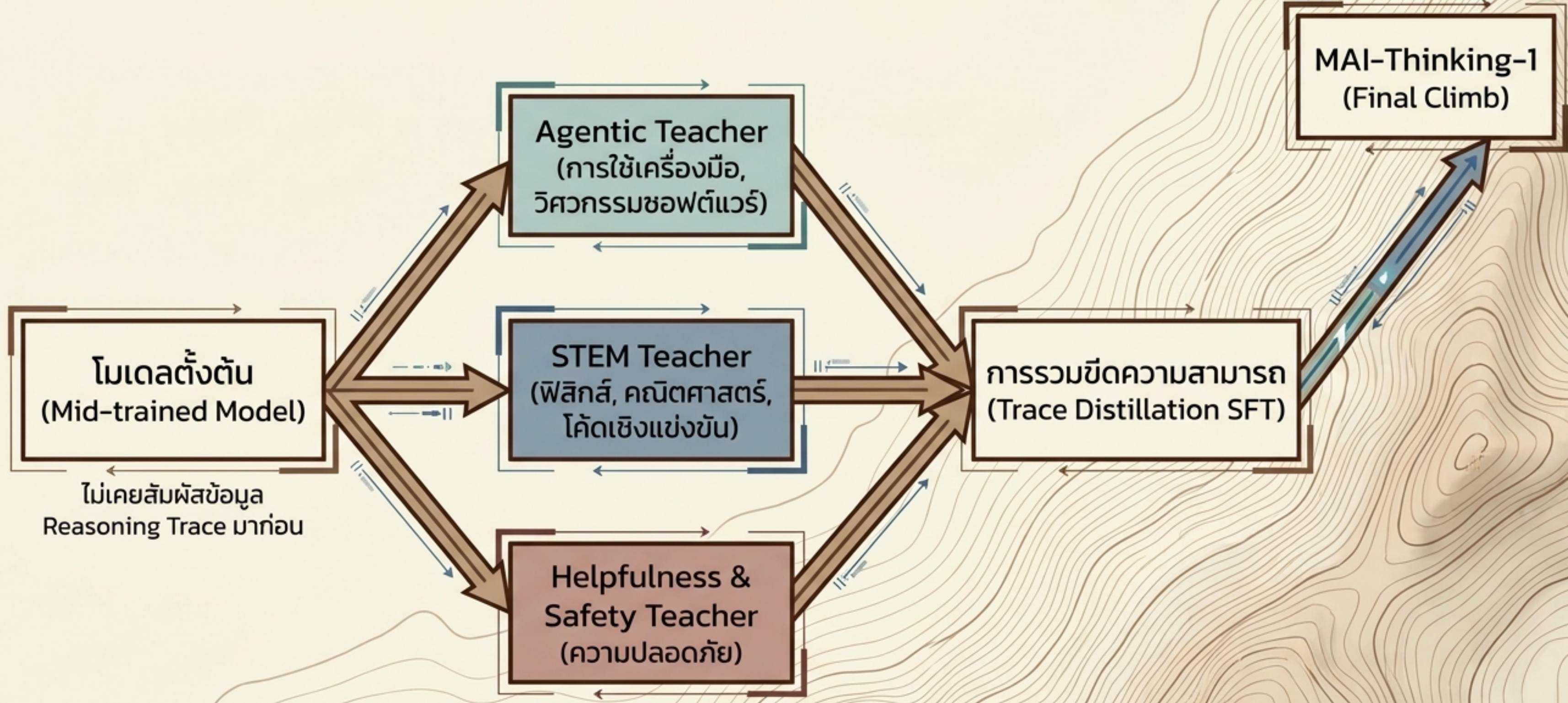
เมื่อสถาปัตยกรรมเปลี่ยน ซอฟต์แวร์ต้องปรับตาม



การรักษา MFU ให้อยู่เหนือ 20% ในทุกช่วงสเกล จำเป็นต้องสร้างเทคโนโลยีใหม่เพื่อคลายคอขวดของระบบเสมอ



# การไต่เขาด้วย Reinforcement Learning

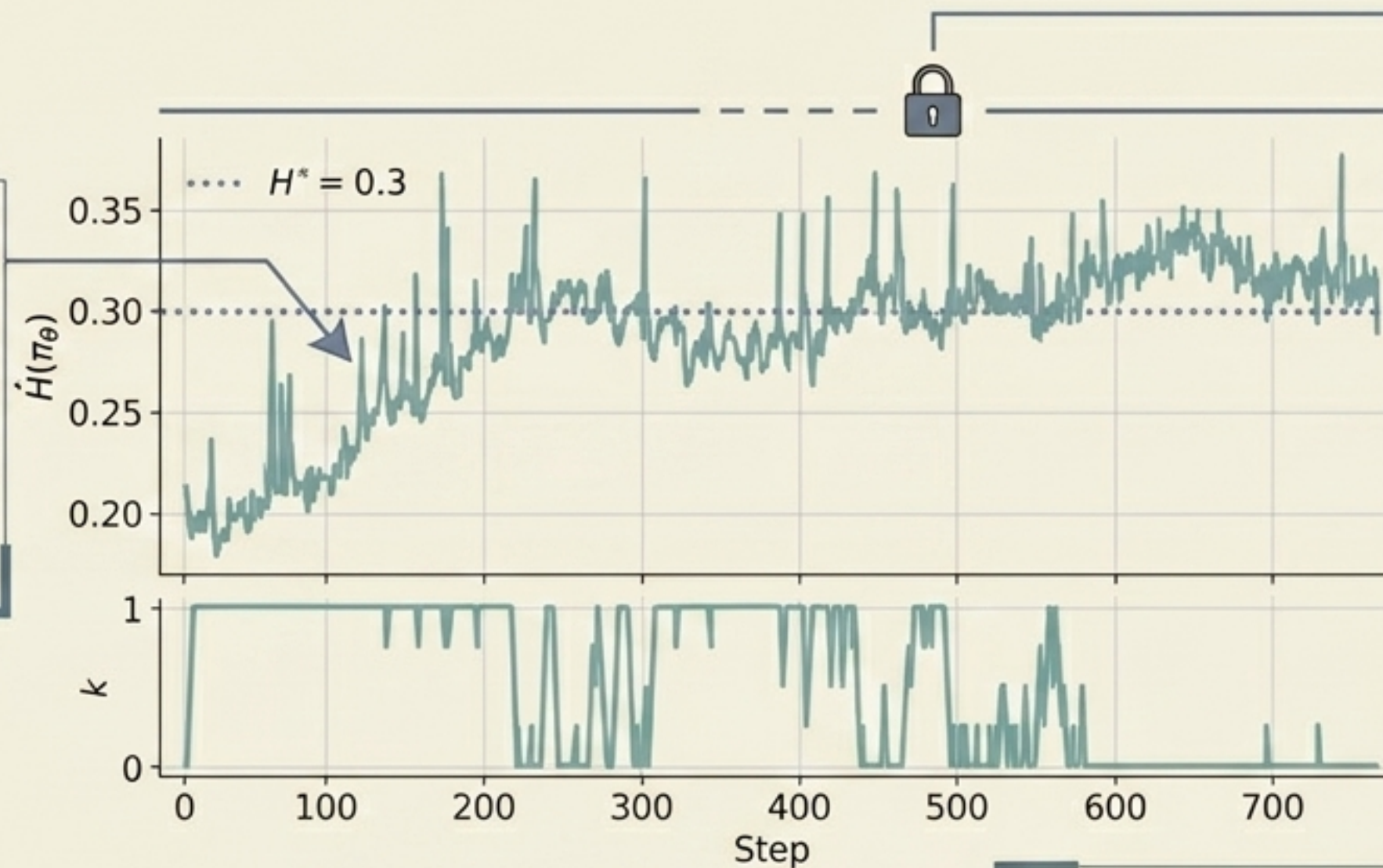




# ยกระดับคณิตศาสตร์: Adaptive GRPO

## Adaptive Entropy Control

ป้องกันโมเดลล่ม (Collapse) ด้วยการปรับขอบเขต Upper Clip แบบไดนามิก. ขยายขอบเขตเมื่อ Entropy ต่ำ และบีบให้แคบเมื่อ Entropy สูงเกินไป



## Outer Ratio Clip

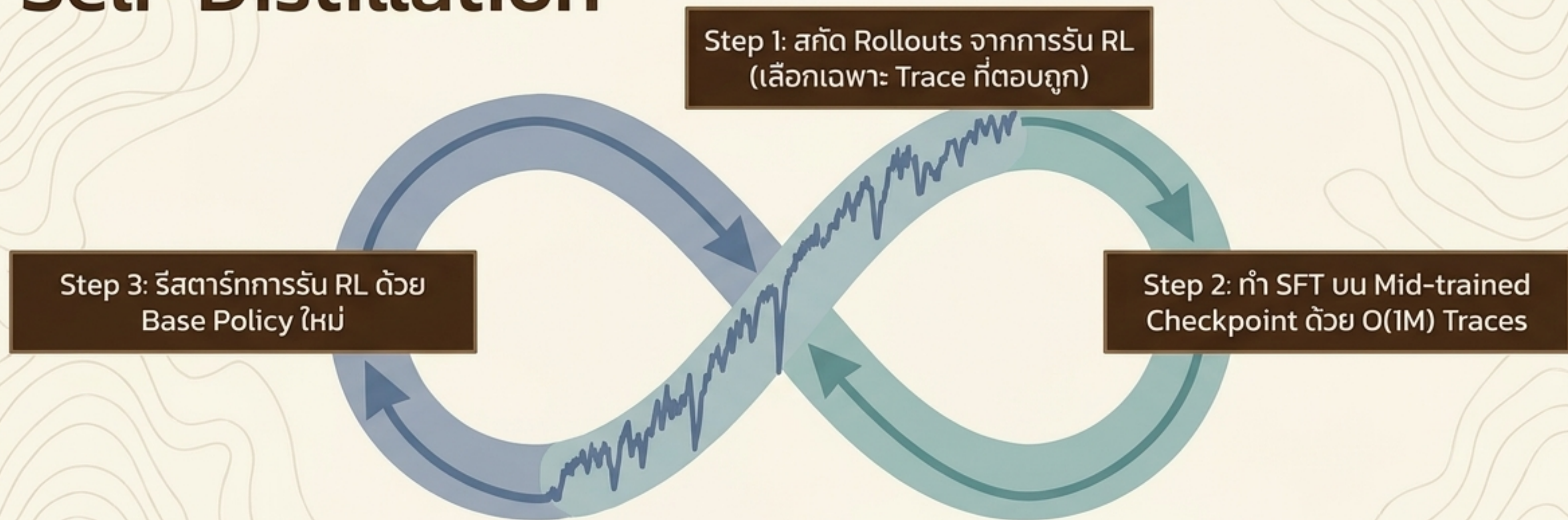
ป้องกันปัญหา Gradient ระเบิด ด้วยการบังคับใช้ Hard Clip (rmax = 50) กับทุก ๆ Branch ของสมการ

## Length Penalty

ควบคุมความฟุ่มเฟือยของคำตอบ (Hedging) โดยผูกบทลงโทษความยาวเข้ากับ "ความยากของคำถาม" คำถามง่ายจะโดนลงโทษหนักหากตอบยาวเกินจำเป็น



# สร้างความเสถียรระยะยาวด้วย Self-Distillation



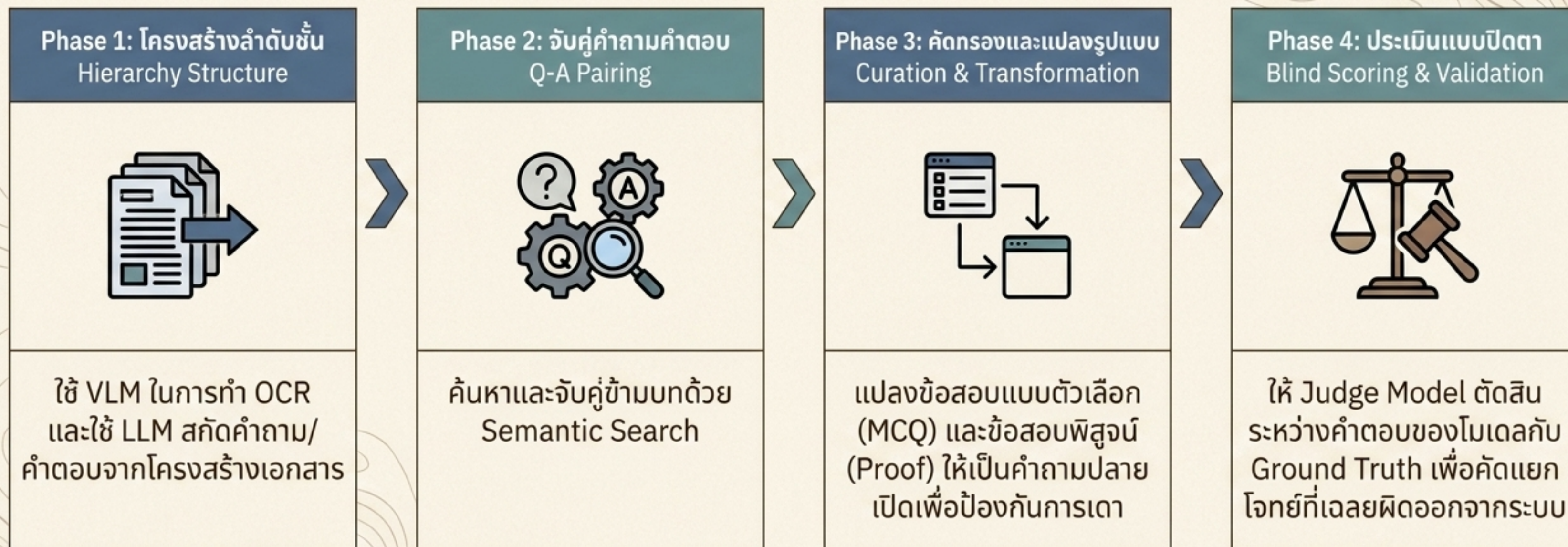
## Insight Box

### ทำไมต้องทำ? (Why Self-Distillation?)

- รีเซ็ตความไม่เสถียรของตัวเลข (Numeric instability) ที่สะสมในระบบ
- ถ่ายโอนความสามารถจากโมเดลรุ่นก่อนหน้า
- เปลี่ยนรูปแบบจาก Prompting ไปสู่ Native Chat Format ได้อย่างแนบเนียน



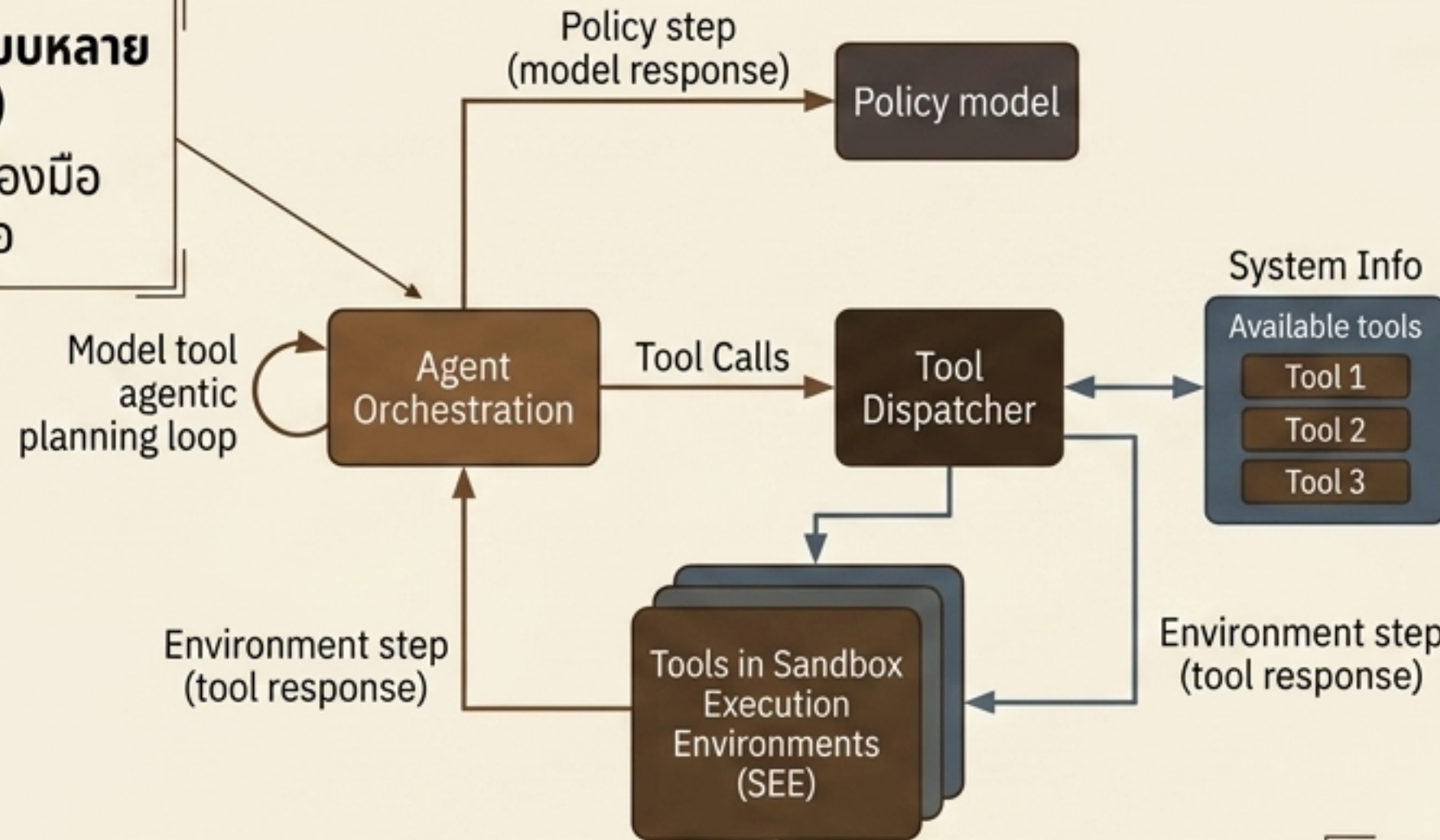
# เส้นทาง STEM: การสร้างข้อมูลที่ตรวจสอบได้ (Verifiable Extraction)





# เส้นทาง Agentic: ระบบปฏิบัติการใน Sandbox (SEE)

**ReAct Loop: วงจรตัดสินใจแบบหลายขั้นตอน (Multi-step Rollout)**  
โมเดลวิเคราะห์คำสั่ง เลือกใช้เครื่องมือ และรอรับผลลัพธ์เพื่อวางแผนต่อ



**Sandbox Execution Environment (SEE):**  
คอนเทนเนอร์ที่แยกอิสระระดับเน็ตเวิร์ก  
ทนทานต่อข้อผิดพลาด (Fault-tolerant)  
ป้องกันสถานะรั่วไหล (State leakage) ระหว่างการรัน

## Verifiable Reward:

ให้รางวัลจากผลลัพธ์ที่จับต้องได้จริง  
(เช่น โค้ดคอมไพล์ผ่าน, Unit test ถูกต้อง,  
หรือสถานะ Database เปลี่ยนแปลงตามคำสั่ง)

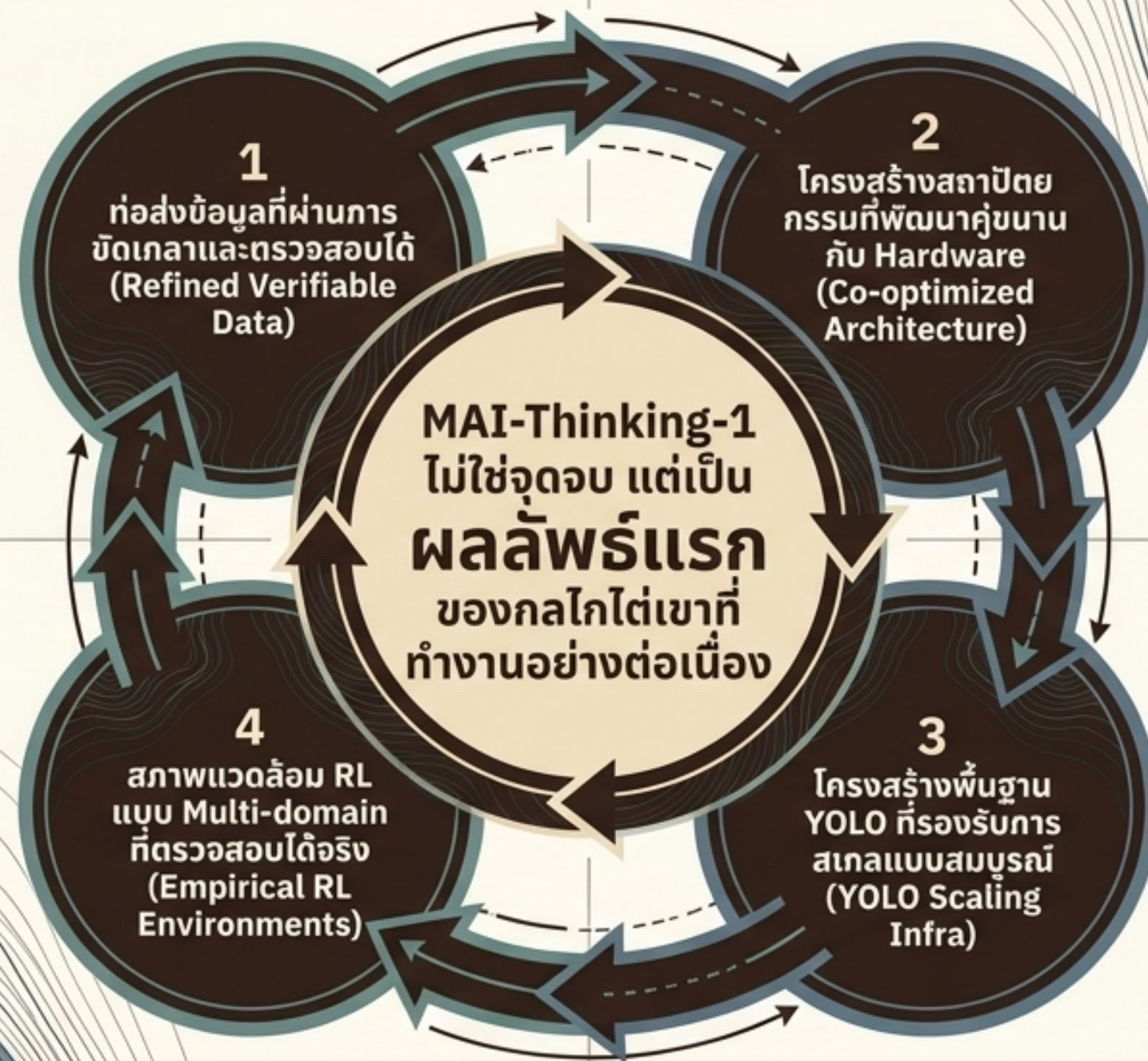


# เปรียบเทียบเส้นทางผู้เชี่ยวชาญทั้ง 3 มิติ

|                | STEM Climb                                     | Agentic Climb                                      | Helpfulness & Safety Climb                                                   |
|----------------|------------------------------------------------|----------------------------------------------------|------------------------------------------------------------------------------|
| กลไกให้รางวัล  | กฎตายตัวทางคณิตศาสตร์<br>เครื่องมือตรวจประเมิน | ผลลัพธ์จากการรันโค้ด /<br>สถานะระบบใน SEE          | AI Judge / กฎระเบียบ<br>ด้านความปลอดภัย                                      |
| ลักษณะ Rollout | การคิดขั้นตอนเดียว<br>(Single-pass CoT)        | การโต้ตอบหลายขั้นตอน<br>(Multi-step Agentic Loop)  | การแชทและสนทนา<br>(Conversational)                                           |
| ความท้าทายหลัก | ภาษาปนเปื้อนใน CoT,<br>การเดาข้อสอบ (Hedging)  | Context ยาวขึ้นเรื่อยๆ,<br>การวางแผนระยะยาวลุ่มหลง | การรักษาสมดุลระหว่าง<br>'การปฏิเสธอย่างปลอดภัย'<br>และ 'การให้ความช่วยเหลือ' |



# บทสรุป: วงจรการเพิ่มประสิทธิภาพเชิงประจักษ์



ความก้าวหน้าของ AI ไม่ได้เกิดจากความบังเอิญ  
แต่เกิดจากระบบการทางวิศวกรรมระดับระบบที่ทำซ้ำได้อย่างแม่นยำ